

Лабораторные работы 4-8 ОСиСП. Теория для ответа

Лабораторная работа №4. Задача производителя-потребителя для процессов

Условие:

Основной процесс создает очередь сообщений, после чего ожидает и обрабатывает нажатия клавиш, порождая и завершая процессы двух типов — производители и потребители.

Очередь сообщений представляет собой классическую структуру — кольцевой буфер, содержащий указатели на сообщения, и пара указателей на голову и хвост.

Помимо этого очередь содержит счетчик добавленных сообщений и счетчик извлеченных.

Производители формируют сообщения и, если в очереди есть место, перемещают их туда.

Потребители, если в очереди есть сообщения, извлекают их оттуда, обрабатывают и освобождают память с ними связанную.

Для работы используются два семафора для заполнения и извлечения, а также мьютекс или одноместный семафор для монопольного доступа к очереди.

Сообщения имеют следующий формат (размер и смещение в байтах):

Имя	Размер	Смещение	Описание
type	1	0	тип сообщения
hash2		1	контрольные данные
size	1	3	длина данных в байтах (от 0 до 256)
data ((size + 3)/4)*4			данные сообщения

Производители генерируют сообщения, используя системный генератор rand(3) для size и data.

В качестве результата для size используется остаток от деления на 257.

Если остаток от деления равен нулю, rand(3) вызывается повторно.

Если остаток от деления равен 256, значение size устанавливается равным 0, реальная длина сообщения при этом составляет 256 байт.

При формировании сообщения контрольные данные формируются из всех байт сообщения.

Значение поля hash при вычислении контрольных данных принимается равным нулю.

Для расчета контрольных данных можно использовать любой подходящий алгоритм на выбор студента.

После помещения значения в очередь перед освобождением мьютекса очереди производитель инкрементирует счетчик добавленных сообщений.

Затем после поднятия семафора выводит строку на stdout, содержащую помимо всего новое значение этого счетчика.

Потребитель, получив доступ к очереди, извлекает сообщение и удаляет его из очереди.

Перед освобождением мьютекса очереди инкрементирует счетчик извлеченных сообщений.

Затем после поднятия семафора проверяет контрольные данные и выводит строку на stdout, содержащую помимо всего новое значение счетчика извлеченных сообщений.

При получении сигнала о завершении процесс должен завершить свой цикл и только после этого завершиться, не входя в новый.

Следует предусмотреть задержки, чтобы вывод можно было успеть прочитать в процессе работы программы.

Следует предусмотреть защиту от тупиковых ситуаций из-за отсутствия производителей или потребителей.

Теория:

Семафор (semaphore) — объект, ограничивающий количество процессов/потоков, которые могут войти в заданный участок кода. Семафоры используются для синхронизации и защиты передачи данных через совместно используемую память, а также для синхронизации работы процессов и потоков. Семафор представляет собой целую переменную, принимающую неотрицательные значения, доступ любого процесса к которой, за исключением момента ее инициализации, может осуществляться только через две атомарные операции.

Кольцевой буфер (ring buffer) - или буфер FIFO (First-In, First-Out), представляет собой структуру данных, в которой элементы данных организованы в виде кольца. Кольцевой буфер имеет фиксированный размер и может хранить определенное количество элементов.

Мьютекс (mutex, mutual exclusion — «взаимное исключение») — аналог одноместного семафора, служащий для синхронизации одновременно выполняющихся потоков. Мьютекс отличается от семафора тем, что только владеющий им поток может его освободить. Мьютексы — это один из вариантов семафорных механизмов для организации взаимного исключения. Мьютексы могут находиться в одном из двух состояний — отмеченном или неотмеченном (открыт и закрыт).

README:

Для создания производителя(зелёный) введите 'p'

Для создания потребителя(красный) введите 'c'

Для удаления производителя введите 'r'

Для удаления потребителя введите 'o'

Для вывода всех процессов в stdout введите 'l'

Для завершения всех процессов и завершения выполнения введите 'q'

Размер буфера - 10

Поле data представляет собой случайную последовательность значений от 0 до 255

Для кодирования hash суммировался каждый элемент data, прибавлялся size и если type равен 1, то сумма умножалась на 10

Если hash не совпал, то выводится сообщение "ERROR" белого цвета

Лабораторная работа № 5. Потоки исполнения, взаимодействие и синхронизация

Условие:

Задача производители-потребители для потоков. Аналогична лабораторной №4, но только с потоками в рамках одного процесса.

Дополнительно обрабатывается еще две клавиши – увеличение и уменьшение размера очереди.

Теория:

То же, что и в лабораторной работе №4.

README

Для создания производителя(зелёный) введите 'p'

Для создания потребителя(красный) введите 'c'

Для удаления производителя введите 'r'

Для удаления потребителя введите 'o'

Для вывода всех процессов в stdout введите 'l'

Для завершения всех процессов и завершения выполнения введите 'q'

Для увеличения размера буфера на 1 введите '+'

Для уменьшения размера буфера на 1 введите '-'

Размер буфера - 10

Поле data представляет собой случайную последовательность значений от 0 до 255

Для кодирования hash суммировался каждый элемент data, прибавлялся size и если type равен 1, то сумма умножалась на 10

Если hash не совпал, то выводится сообщение "ERROR" белого цвета

Лабораторная работа № 6. Работа с файлами, отображенными в память

Условие:

Написать многопоточную программу `sort_index` для сортировки вторичного индексного файла таблицы базы данных, работающую с файлом в двух режимах: `read()/write()` и с использованием отображение файлов в адресное пространство процесса.

Программа должна запускаться следующим образом:

`sort_index memsize granul threads filename`

Параметры командной строки:

`memsize` := размер рабочего буфера, кратный размеру страницы (`getpagesize()`) `blocks` :=

порядок разбиения буфера

`threads` := количество потоков (от `k` до `N`)

`k` := количество ядер

`N` := максимальное количество потоков (8k??) `filename` := имя файла

Количество блоков должно быть степенью двойки и превышать количество потоков.

Для целей тестирования написать программу генерации неотсортированного индексного файла.

Теория:

1. `pthread_create()`: Создает новый поток и начинает его выполнение.
2. `pthread_exit()`: Завершает вызывающий поток и освобождает его ресурсы.
3. `pthread_join()`: Блокирует вызывающий поток до тех пор, пока указанный поток не завершится.
4. `pthread_yield()`: Передает управление другому потоку с тем же приоритетом. Операционная система выбирает следующий поток для выполнения.
5. `pthread_cancel()`: Завершает указанный поток немедленно.
6. `pthread_barrier_init()`: Инициализирует барьер с указанным количеством потоков, которые должны достичь барьера, прежде чем все они будут разблокированы.
7. `pthread_barrier_destroy()`: Уничтожает барьер, освобождая связанные с ним ресурсы.
8. `pthread_barrier_wait()`: Блокирует поток, пока все потоки, связанные с барьером, не достигнут барьера. Когда все потоки достигают барьера, они разблокируются одновременно.
9. `mmap()`: Создает отображение файла в памяти или анонимное отображение памяти.
В случае успеха вызов `mmap()` возвращает расположение отображаемой области.
При ошибке он возвращает `MAP_FAILED` и устанавливает `errno` соответствующее значение.

Вызов `mmap()` никогда не возвращает 0.

При вызове `mmap()` мы указываем ядру отобразить в память `len` байт объекта, представленного файловым дескриптором `fd` и начинающийся в файле со смещения в `offset` байт.

Если включить `addr`, то данное значение указывает адрес в памяти, который имеет приоритет при выборе начальной точки отображения.

`MAP_FIXED` - параметр `addr()` обязателен

`MAP_PRIVATE` - отображение можно использовать совместно

`MAP_SHARED` - отображение нельзя использовать совместно

Преимущества `mmap()`:

- При чтении файла, отображенного в память, или при его записи мы обходимся без лишнего копирования, которое выполняется при применении системных вызовов `read()` или `write()`, требующих переноса информации в пользовательский буфер и из него.
- Кроме возможных отказов страниц при записи в файл, отображаемый в память, и при его считывании нет никаких издержек, связанных с дополнительными системными вызовами и переключением контекста.
- Когда несколько процессов отображают в память один и тот же объект, информация из этого объекта совместно используется всеми процессами.
- Позиционирование в ходе отображения сводится к обычным манипуляциям с указателем. Системный вызов `lseek()` в данном случае не нужен.

Недостатки `mmap()`:

- По своей величине отображения в памяти всегда являются целочисленными кратными размеру страницы, поэтому разница между размером базового файла и целым количеством страниц приходится на напрасно расходуемое пустое место.
- Отображения в памяти должны вписываться в адресное пространство процесса.
- Возникают издержки, связанные с созданием и поддержкой отображений, находящихся в памяти, и ассоциированных с ними структур данных, расположенных в ядре.

10. **`munmap()`**: Удаляет отображение памяти, ранее созданное с помощью `mmap()`.

- В Linux предоставляется вызов `munmap()`, позволяющий удалять отображение, созданное при вызове `mmap()`. При успехе `munmap()` возвращает 0. При ошибке он возвращает -1 и присваивает `errno` соответствующее значение.

я юзал:

`pthread_exit()`

233 stroka main

pthread_barrier_init()	266
pthread_barrier_wait()	
pthread_barrier_destroy()	326

Лабораторная работа No 7. Блокировки чтения/записи и условные переменные

Условие:

Здесь две программы.

1) Задача «производители-потребители». Аналогична лабораторной No 4, но для потоков с использованием условных переменных.

Изучаемые системные вызовы (префикс pthread_ опущен): cond_init(), cond_destroy(), cond_*wait(), cond_signal().

2) Конкурентный доступ к совместно используемому файлу, используя блокировку чтения-записи. Изучаемые системные вызовы: fcntl(F_GETLK, F_SETLK, F_SETLKW, F_UNLK).

Программа в режиме конкурентного доступа читает из и пишет в файл, содержащий записи фиксированного формата. Формат записей произвольный. Примерный формат записи:

```
struct record_s {  
    char name[80]; // Ф.И.О. студента  
    char address[80]; // адрес проживания  
    uint8_t semester; // семестр  
}
```

Файл должен содержать не менее 10 записей. Создается и наполняется с помощью любых средств.

Программа должна выполнять следующие операции:

1) LST - Отображение содержимого файла с последовательной нумерацией записей 2) GET(Rec_No) - получение записи с порядковым номером Rec_No;

3) Модификацию полей записи

4) PUT() - сохранение последней прочитанной и модифицированной записи по месту. Интерфейс с пользователем на «вкус» студента.

Теория:

1) 1. pthread_cond_init(): Инициализирует условную переменную. Она должна быть вызвана перед использованием любой конкретной условной переменной.

2. pthread_cond_destroy(): Уничтожает условную переменную и освобождает ресурсы, связанные с ней.

3. pthread_cond_wait(): Блокирует вызывающий поток до получения сигнала от другого потока. При вызове этой функции, вызывающий поток освобождает мьютекс (чаще всего вместе с ним), и он блокируется до получения сигнала. После получения сигнала поток снова захватывает мьютекс и продолжает выполнение.

4. pthread_cond_signal(): Отправляет сигнал одному из ожидающих потоков, связанных с указанной условной переменной. Если есть несколько потоков, ожидающих условную переменную, то сигнал может быть доставлен только одному из них.

Поток, вызывающий pthread_cond_wait(), будет ожидать, пока другой поток не отправит сигнал с помощью pthread_cond_signal(), указывая на изменение состояния или достижение определенного условия.

2) 1. ``fcntl(F_GETLK)``: Получает информацию о блокировке, которая установлена на файловый дескриптор или диапазон файловых дескрипторов. Если блокировка не установлена, возвращается информация о свободной блокировке.

2. ``fcntl(F_SETLK)``: Устанавливает блокировку на файловом дескрипторе или диапазоне файловых дескрипторов. Если блокировка успешно установлена, она будет предотвращать другие процессы от получения блокировки на том же диапазоне.

3. ``fcntl(F_SETLKW)``: Устанавливает блокировку на файловом дескрипторе или диапазоне файловых дескрипторов. Если блокировка уже установлена другим процессом, вызывающий процесс будет заблокирован до тех пор, пока блокировка не будет освобождена.

4. ``fcntl(F_UNLK)``: Освобождает блокировку, установленную на файловом дескрипторе или диапазоне файловых дескрипторов.

Функция ``fcntl`` предоставляет механизм управления блокировками файловых ресурсов и используется для синхронизации доступа к файлам между разными процессами или потоками.

README

Введите "LST" для вывода списка студентов;

Введите "GET(num)" для того, чтобы начать работать со студентом под номером num;

Введите "MODIFY" для того, чтобы сгенерировать нового студента;

Введите "PUT()" для того, чтобы заменить взятого студента на сгенерированного студента.

Лабораторная работа No 8. Сокеты. Взаимодействие процессов.

Условие:

Задача - разработка многопоточного сервера и клиента, работающих по простому протоколу.

Изучаемые системные вызовы: `socket()`, `bind()`, `listen()`, `connect()`, `accept()` и прочих, связанных с адресацией в домене `AF_INET`.

Протокол должен содержать следующие запросы:

`ECHO` - эхо-запрос, возвращающий без изменений полученное от клиента;

`QUIT` - запрос на завершение сеанса;

`INFO` - запрос на получения общей информации о сервере;

`CD` - изменить текущий каталог на сервере;

`LIST` - вернуть список файловых объектов из текущего каталога.

Протокол может содержать дополнительные запросы по выбору студента, не выходящие за пределы корневого каталога сервера и не изменяющих файловую систему в его дереве.

Запросы клиенту отправляются на `stdin`.

Ответы сервера и ошибки протокола выводятся на `stdout`.

Ошибки системы выводятся на `stderr`.

Подсказка клиента для ввода запросов - символ '>'.

Клиент помимо интерактивных запросов принимает запросы из файла. Файл с запросами указывается с использованием префикса '@':

Теория:

Сокет — это обобщенный канал межпроцессного взаимодействия.

Как и канал, сокет представлен как файловый дескриптор.

Однако, в отличие от каналов, сокеты поддерживают связь между несвязанными процессами и даже между процессами, запущенными на разных машинах, которые обмениваются данными по сети.

AF_INET

Обозначает формат адреса, который соответствует пространству имен Internet. Имя этого пространства имен - **PF_INET**.

Системные вызовы:

1. `socket()`: Создает новый сокет и возвращает его дескриптор. Этот вызов определяет домен сокета, тип сокета и протокол.

2. `bind()`: Привязывает сокет к конкретному адресу IP и порту на локальной машине. Этот вызов используется для серверных сокетов, чтобы указать, на каком адресе и порту сокет будет прослушивать входящие соединения.

3. `listen()`: Устанавливает сокет в режим прослушивания входящих соединений. Этот вызов применяется к серверному сокету после вызова `bind()`.

4. ``connect()``: Устанавливает соединение с удаленным сервером. Этот вызов применяется к клиентскому сокету и используется для установки связи с сервером по указанному адресу IP и порту.

5. ``accept()``: Принимает входящее соединение на серверном сокете, создавая новый сокет для обмена данными с клиентом. Этот вызов блокирует выполнение программы до тех пор, пока не будет получено новое входящее соединение.

6. ``send()``, ``recv()``: Используются для отправки и приема данных по сети через установленное соединение. ``send()`` отправляет данные на сокет, а ``recv()`` принимает входящие данные.

7. ``inet_addr()``: Преобразует строку с адресом IPv4 в структуру ``in_addr``, которая представляет адрес в сетевом порядке байт.

8. ``inet_ntoa()``: Преобразует структуру ``in_addr`` с адресом IPv4 из сетевого порядка байт в строку.